REPORT DI OSSERVAZIONE

WIRESHARK PER OSSERVARE L'HANDSHAKE A 3 VIE TCP

OBIETTIVO

In questo laboratorio ho utilizzato Wireshark per catturare ed esaminare i pacchetti generati tra il browser di un host e un server web, osservando in particolare *l'handshake* a tre vie con cui TCP stabilisce una sessione affidabile prima di iniziare la comunicazione HTTP. Ho inoltre utilizzato tcpdump da terminale per visualizzare gli stessi pacchetti catturati.

Il laboratorio si è svolto in un ambiente Mininet con topologia CyberOps, composta dal router R1, dallo switch S1 e dagli host *H1, H2, H3* (rete 10.0.0.0/24) e *H4* (rete 172.16.0.0/12), quest'ultimo usato come server web di destinazione.

SVOLGIMENTO

PARTE 1: PREPARARE GLI HOST PER CATTURARE IL TRAFFICO

Ho avviato la VM CyberOps e ho effettuato l'accesso con l'utente analyst. Ho poi avviato Mininet eseguendo lo script cyberops_topo.py, che ha costruito la topologia con R1, S1 e gli host H1-H4.



Figura 1: Topologia di rete CyberOps avviata in Mininet

Dalla CLI di Mininet ho aperto i terminali xterm per gli host H1 e H4, e ho avviato il server web sull'host H4 tramite lo script reg_server_start.sh.

Su H1, per motivi di sicurezza non è possibile eseguire Firefox da root, quindi sono passato all'utente analyst con il comando su analyst e ho avviato il browser web con firefox &. Ho poi avviato una sessione tcpdump sull'interfaccia H1-eth0, inviando l'output al file capture.pcap e limitando la cattura a 50 pacchetti con

l'opzione -c 50. Dopo l'avvio della cattura, ho navigato rapidamente all'indirizzo 172.16.0.40 dal browser, raggiungendo la pagina di benvenuto del server web nginx in esecuzione su H4.

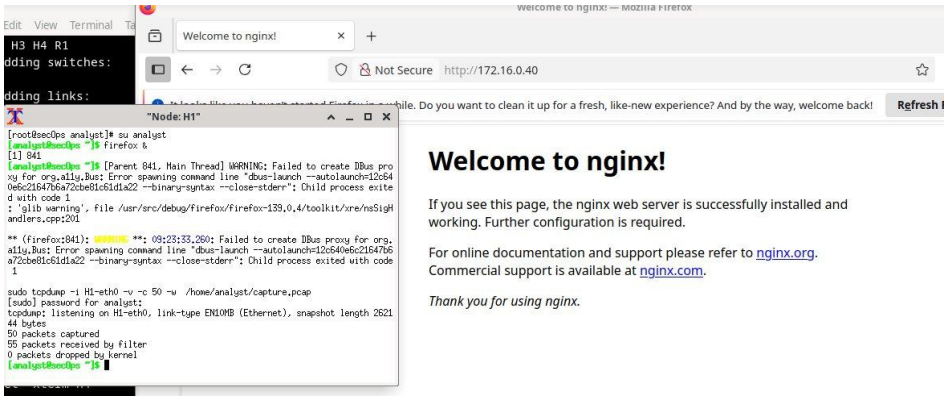


Figura 2: Terminale di H1 con Firefox e tcpdump avviati, e la pagina di benvenuto di nginx raggiunta su 172.16.0.40

La cattura si è fermata automaticamente al raggiungimento dei 50 pacchetti, come si vede dal messaggio “50 packets captured” mostrato nel terminale.

PARTE 2: ANALIZZARE I PACCHETTI CON WIRESHARK

Ho avviato Wireshark sull'host H1 e ho aperto il file di cattura salvato in /home/analyst/capture.pcap tramite File > Open. Ho poi applicato il filtro tcp alla cattura, che mi ha permesso di isolare i primi quattro frame relativi al traffico di interesse: i tre pacchetti dell'handshake a tre vie (SYN, SYN-ACK, ACK) seguiti dalla richiesta HTTP GET.

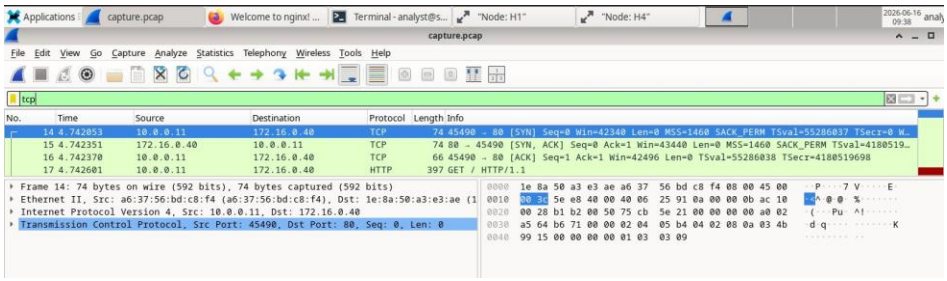


Figura 3: Wireshark con il filtro tcp applicato; in evidenza il primo pacchetto dell'handshake (SYN) con i relativi dettagli TCP

Esaminando il primo pacchetto (il SYN iniziale) ho trovato i seguenti dettagli: la porta TCP di origine è 45490, una porta alta assegnata dinamicamente dal sistema operativo al momento della connessione; quindi, si classifica come porta effimera o dinamica, non una porta nota o registrata. La porta di destinazione è 80, la porta ben nota (well-known) riservata al protocollo HTTP. Il flag impostato è SYN (0x002), che segnala l'inizio della richiesta di connessione, e il numero di sequenza relativo è impostato a 0, il punto di partenza della numerazione per questa connessione.

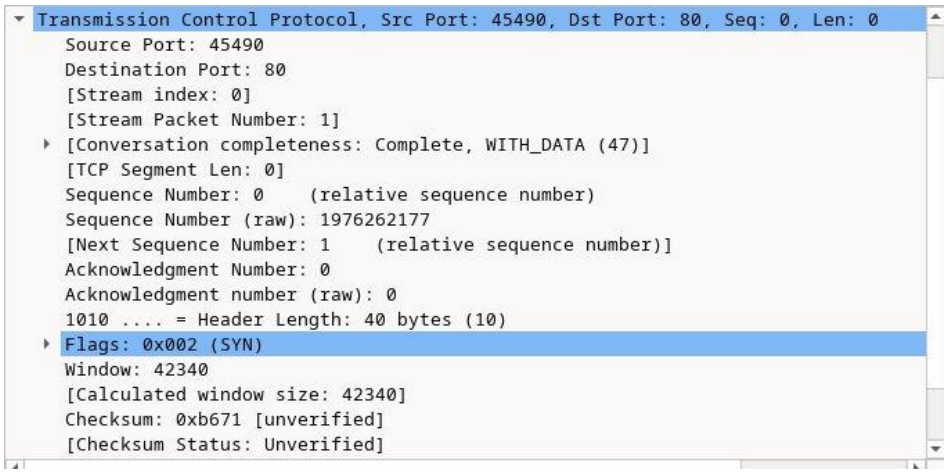


Figura 4: Dettagli del primo pacchetto dell'handshake (SYN), porta di origine 45490, porta di destinazione 80

Il secondo pacchetto è la risposta del server web alla richiesta iniziale di avviare una sessione. Le porte di origine e destinazione sono invertite rispetto al primo pacchetto (origine 80, destinazione 45490), i flag impostati sono SYN e ACK (0x012) e i numeri relativi di sequenza e acknowledgment sono impostati rispettivamente a 0 e 1.

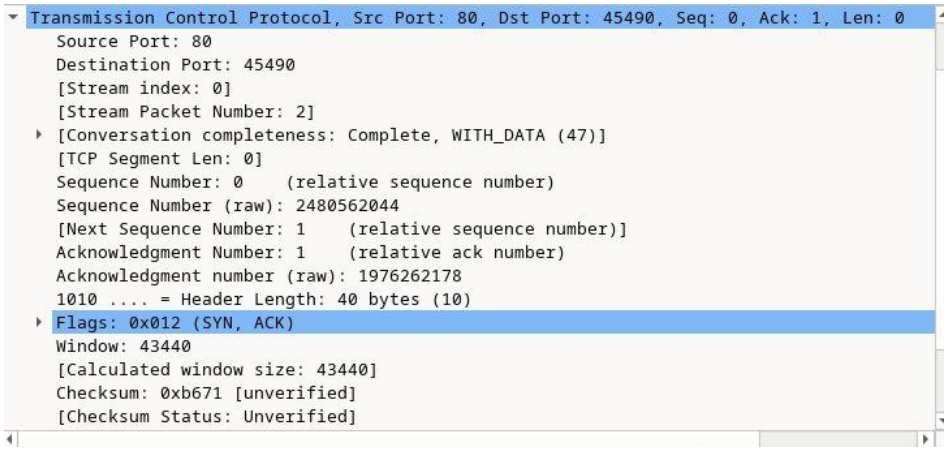


Figura 5: Dettagli del secondo pacchetto dell'handshake (SYN, ACK), porta di origine 80, porta di destinazione 45490

Il terzo e ultimo pacchetto dell'handshake ha il flag ACK (0x010) impostato. I numeri relativi di sequenza e acknowledgment sono entrambi impostati a 1 come punto di partenza: a questo punto la connessione TCP è stabilita e la comunicazione tra l'host di origine e il server web può iniziare.

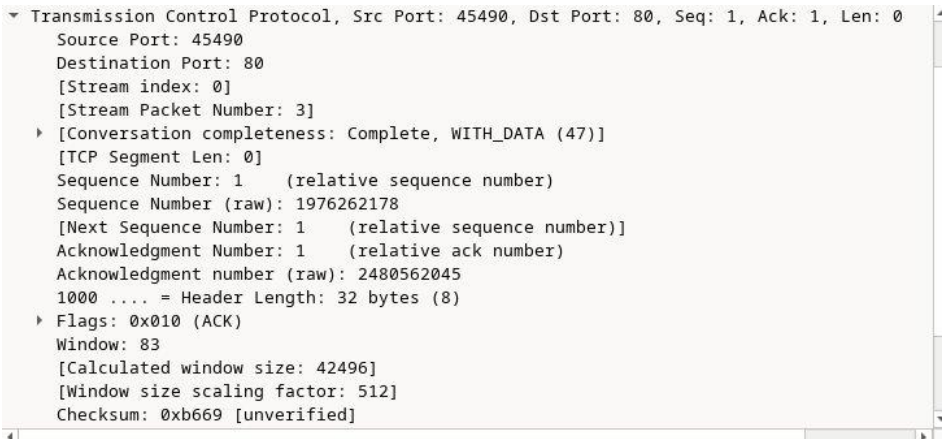


Figura 6: Dettagli del terzo pacchetto dell'handshake (ACK), con sequence e acknowledgment number relativi entrambi a 1

PARTE 3: VISUALIZZARE I PACCHETTI CON TCPDUMP

Ho verificato che è possibile visualizzare lo stesso file di cattura anche da terminale, filtrando le informazioni desiderate con tcpdump. Per capire il significato delle varie opzioni del comando ho consultato la pagina di manuale con man tcpdump, cercando in particolare la descrizione dell'opzione -r.

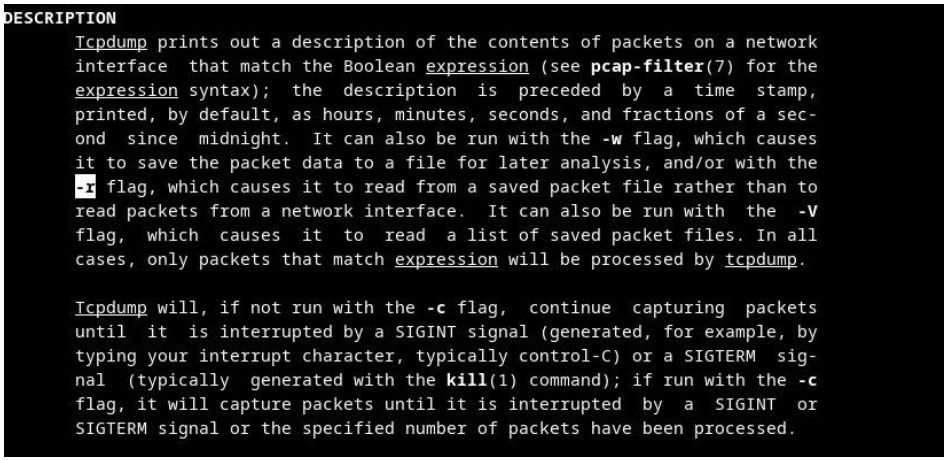


Figura 7: Descrizione dell'opzione -r nella pagina di manuale di tcpdump

Dalla descrizione ho capito che l'opzione -r serve a far leggere a tcpdump i pacchetti da un file di cattura salvato in precedenza, invece di acquisirli in tempo reale da un'interfaccia di rete.

Ho quindi usato questo comando per visualizzare i primi 3 pacchetti TCP del file di cattura:

```
tcpdump -r /home/analyst/capture.pcap tcp -c 3
```

L'output mostra esattamente gli stessi tre pacchetti dell'handshake già analizzati in Wireshark: il SYN iniziale (flag [S]) dalla porta 45490 alla porta 80, la risposta SYN-ACK (flag [S.]) del server, e l'ACK finale (flag [.]) che conclude la negoziazione, confermando che l'handshake a tre vie è stato completato correttamente.

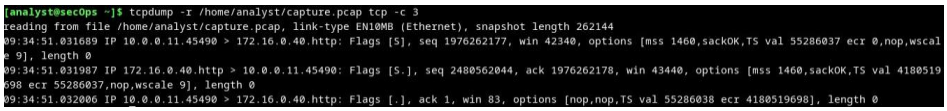


Figura 8: Output di tcpdump -r capture.pcap tcp -c 3, con i tre pacchetti dell'handshake TCP

Per concludere il laboratorio sono tornato al terminale principale di Mininet, ho terminato la sessione con il comando quit e ho pulito i processi avviati da Mininet con sudo mn -c, inserendo la password richiesta.

DOMANDE DI RIFLESSIONE

1. TRE FILTRI WIRESHARK UTILI A UN AMMINISTRATORE DI RETE

Un amministratore di rete potrebbe trovare utili filtri come *ip.addr == x.x.x.x*, per isolare tutto il traffico, sia in entrata che in uscita, verso o da un host specifico, ad esempio per indagare su un dispositivo sospetto. Un altro filtro utile è *tcp.port == 80* (oppure semplicemente http), per isolare il traffico web e individuare eventuali rallentamenti o errori su quel servizio. Infine, filtri come dns o icmp sono utili rispettivamente per il troubleshooting della risoluzione dei nomi e per verificare problemi di connettività di base, come quelli evidenziati da un comando ping. Anche filtri più generali come tcp o udp restano comunque utili come primo livello di separazione tra i due protocolli di trasporto, prima di applicare filtri più specifici.

2. ALTRI USI DI WIRESHARK IN UNA RETE DI PRODUZIONE

Wireshark da solo non è un vero sistema **IDS** (Intrusion Detection System), perché quelli sono strumenti automatici come Snort o Suricata che analizzano il traffico in tempo reale e in modo continuativo. In una rete di produzione, Wireshark viene però usato soprattutto per il troubleshooting puntuale, ad esempio per capire perché una connessione è lenta o si interrompe, e per l'analisi forense dopo un incidente di sicurezza, ricostruendo nel dettaglio cosa è successo a partire dal traffico catturato. Viene inoltre utilizzato per verificare che le comunicazioni siano effettivamente cifrate e non viaggino in chiaro, e per controllare che firewall e altri apparati di rete stiano applicando correttamente le regole previste.

CONCLUSIONI

Questo laboratorio mi ha permesso di vedere concretamente come funziona l'handshake a tre vie di TCP, analizzando passo per passo i tre pacchetti (SYN, SYN-ACK, ACK) sia con Wireshark che con tcpdump da terminale. Ho capito meglio la differenza tra porte note, come la porta 80 usata da HTTP, e porte effimere, assegnate dinamicamente dal sistema al client per la durata della connessione. Ho inoltre approfondito l'utilizzo pratico di Wireshark e tcpdump come strumenti complementari: il primo più visuale e adatto all'analisi dettagliata, il secondo più rapido per controlli da riga di comando, entrambi utili in scenari reali di troubleshooting e analisi del traffico di rete.